CSCE441101 Fall 2026

Project : RescueMesh

Phase 0: Distributed Election Algorithms Survey

Prof. Amr El Kadi

Hanna Mostafa 900222857

Joudy El Gayar 900222142

Youssef Badawy 900214672

Sara Faheem 900231240

Abdallah Afifi 900225283

Department of Computer Science and Engineering, AUC

Table of Contents :

1. Introduction and Scope 3

2. Background 4

3. Survey Methodology 6

4. Major Techniques: Brief, Pros and Cons 7

5. Recent Literature 15

6. Comparative Summary 18

7. Projected Techniques for RescueMesh and Justification 23

8. References 33

1. Introduction and Scope

We select a Metric-Weighted Bid Election (MWBE) to choose the executing node for every incident request, and a majority-acknowledged, epoch-fenced Bully election to choose a stable coordinator. The rest of this report explains why.

RescueMesh is a three-node peer cloud. Clients multicast each incident request to all three nodes, and the nodes must agree on which one executes it. One node is periodically silenced for up to 20 s, the other two must keep serving, and the revived node must catch up before it participates again.

Election is therefore needed for two jobs with very different profiles:

Job	**How often**	**What matters most**	**Selection key**
Per-request executor selection	Thousands of times per experiment	Low latency, low message overhead, load awareness	Dynamic load score
Stable coordinator (fault-injection scheduling, recovery donor, lease arbitration)	Rarely, after failures	Safety: never two coordinators acting at once	State Version + node ID (epoch for fencing)

The two jobs have different requirements, so we evaluated each technique against both instead of looking for a single winner. Section 2 gives the background and system assumptions, and Section 3 explains how we searched for sources. Section 4 summarizes the main election techniques and their trade-offs, and Section 5 reviews recent IEEE and ACM work. In Section 6 we compare the techniques on the five criteria from the project specification (assumptions, message complexity, failure behavior, convergence, and suitability), using message counts worked out for our three-node cluster. Section 7 presents the design we selected and why, including where we deliberately avoid election in the Phase 2 peer-to-peer layer.

2. Background

2.1 The election problem

A set of processes with unique identities must agree on exactly one coordinator. The standard definition has two properties:

- **Safety:** at most one process acts as leader at a time. In practice, all correct processes that decide must decide on the same leader.
- **Liveness:** every election eventually ends with a leader chosen among the non-failed processes.

Election is classically triggered when the current coordinator is suspected to have failed. RescueMesh also needs election per task, which makes it a form of distributed **extrema finding**: every node proposes a value (its suitability) and the best value wins.

2.2 System-model assumptions

Dimension	Options	Why it matters	RescueMesh
Timing	Synchronous, partially synchronous, asynchronous	FLP (1985) shows deterministic consensus is impossible in a fully asynchronous system with one crash, so practical election relies on timeouts or randomization.	Partially synchronous LAN; timeouts usable but not perfectly accurate
Topology	Ring, tree, complete graph, arbitrary graph	Drives message complexity	Complete graph of 3 cloud nodes
Identity	Unique IDs vs anonymous	Anonymous networks need randomized symmetry breaking (Itai and Rodeh)	Unique, static node IDs

Failure model	Crash-stop, crash-recovery, omission, Byzantine	Determines whether a node can return with stale state or stale authority	Injected send/receive omission of a live process for up to 20 s, then recovery. Byzantine behavior is out of scope.
Links	Reliable FIFO vs lossy or duplicating	Determines whether election messages need retries and deduplication	TCP/UDP over LAN; delays and occasional loss possible; duplicates possible from client retries

The injected failure deserves emphasis. A silenced node is alive, keeps its memory and its clock, and may still believe it holds a role. When it returns it can act on stale authority unless the protocol fences it. This is harder than crash-stop and drives several choices in Section 7.

2.3 Failure detection

Election can only be as good as failure detection. Chandra and Toueg formalized unreliable failure detectors by **completeness** (every crashed process is eventually suspected) and **accuracy** (correct processes are not wrongly suspected, at least eventually). The **Ω** detector, under which all correct processes eventually trust the same correct process, is the weakest detector that makes consensus solvable. Practical leader election implements Ω with heartbeats and timeouts.

Fixed timeouts force a trade-off: short timeouts give fast failover but false suspicions under load; long timeouts give stability but slow failover. Adaptive detectors such as the **φ-accrual detector** (Hayashibara et al., 2004) instead output a suspicion level computed from observed heartbeat inter-arrival times. This suits RescueMesh, where CPU-heavy planning jobs can delay heartbeats.

2.4 Cost metrics used in this survey

Each technique is compared on: messages per election, time to converge (rounds or round trips), behavior when nodes fail during and after election, timing and topology assumptions, the selection key (static ID or dynamic metric), and implementation complexity in Rust. For reference, comparison-based election on a ring needs Ω(n log n) messages (Burns, 1980;

Frederickson and Lynch, 1987), so O(n log n) ring algorithms are message-optimal. At n = 3, however, constants and round trips matter far more than asymptotics.

2.5 Election, consensus and leases

Modern replicated systems (Paxos, Raft, Zab, Viewstamped Replication) embed leader election inside a consensus protocol. There the leader orders updates, and a **majority quorum** guarantees that two leaders of the same term cannot both succeed. A **lease** additionally lets a leader act alone for a bounded time, trading a clock-drift assumption for fewer messages. A **fencing token** (an epoch or term number attached to every action) lets other nodes reject commands from a leader whose authority has been superseded.

These three ideas, quorum, lease and fencing, are the tools for making the coordinator role safe against a silenced node that returns. They are separable from the full consensus machinery, which Section 7 relies on.

3. Survey Methodology

The survey combined three sources: the course textbook's coordination and election material, the seminal papers each technique originates from, and keyword searches of **IEEE Xplore** and the **ACM Digital Library** filtered to 2020–2026 for recent work.

Search strings (run in both libraries, title/abstract/keyword fields):

#	Query	Purpose
Q1	"leader election" AND ("distributed system" OR "distributed computing")	Broad coverage
Q2	"leader election" AND (Raft OR Paxos OR consensus) AND (failover OR timeout OR availability)	Failover speed and stability
Q3	("bully algorithm" OR "ring algorithm" OR "election algorithm") AND (improved OR modified OR efficient)	Classical-algorithm variants
Q4	"leader election" AND ("load balancing" OR "load-aware" OR "performance-aware" OR priority)	Metric-based selection

Q5	“leader election” AND (“partial network failure” OR partition OR “packet loss”)	Behavior under omission failures
Q6	“leader election” AND (“message complexity” OR “fault-tolerant”)	Theoretical bounds

Inclusion criteria: Peer-reviewed work at an IEEE or ACM venue (journals, conferences, or workshops) that addresses small crash-recovery clusters, failover latency, behavior under partial failures, or selection by measured performance. A few non-IEEE/ACM papers are cited only where they are the direct origin of a technique we borrow; they are labeled as such in Section 5.

Exclusion criteria: Byzantine and proof-of-work leader selection (outside our fault model), and protocols specific to wireless sensor networks or radio-level ad hoc routing.

Screening: Titles and abstracts were screened first; papers that passed were read for their assumptions, failure model, and evaluation method. We recorded whether each evaluation used real deployments or only simulation, because RescueMesh will be measured on real machines.

4. Major Techniques: Brief, Pros and Cons

4.1 Bully algorithm (Garcia-Molina, 1982)

How it works : Nodes have totally ordered IDs. A node that suspects the coordinator has failed sends ELECTION to every higher-ID node. If no higher node answers before a timeout, it broadcasts COORDINATOR and takes over. If one answers, that node continues the election. The highest-ID live node always wins.

Cost: Best case $n - 1$ messages (the highest live node notices first); worst case $O(n^2)$ (the lowest node notices first and every higher node starts its own round). Time: one timeout plus a round trip.

Pros:

- Simple to implement and explain; maps directly onto a complete graph.
- Tolerates further crashes during the election.

- The ID is only a comparison key, so any totally ordered value can replace it, including a load score.

Cons:

- Correctness relies on timeouts being accurate, which assumes a synchronous system. A node that is slow but still alive may be wrongly declared failed.
- When a high-ID node recovers, it immediately takes back leadership, so leadership changes every time the silenced node returns.
- During a partition or while a node is silenced, two coordinators can exist at once, since no majority agreement is required.
- Selection is based on fixed IDs, not on current load.

4.2 Ring algorithms

How it works: Nodes are arranged in a logical ring, and each node knows only its successor. In the basic version (LeLann, 1977), every node sends its ID all the way around the ring, so each node sees every ID and picks the highest. Chang–Roberts (1979) improves on this by having each node forward an ID only if it is higher than its own and drop it otherwise; the node whose own ID makes it back around the ring is the leader. Hirschberg–Sinclair (1980) instead lets each node probe in both directions over distances that double each round. Peterson (1982) and Dolev–Klawe–Rodeh (1982) eliminate candidates in rounds of neighbor comparisons on a one-way ring. In the textbook variant, the initiator circulates an ELECTION message that collects the IDs of all live nodes, then picks the highest and circulates a COORDINATOR message to announce the result.

Cost: LeLann's version always needs $O(n^2)$ messages. Chang–Roberts reduces the average to $O(n \log n)$, although its worst case is still $O(n^2)$. Hirschberg–Sinclair, Peterson and Dolev–Klawe–Rodeh guarantee $O(n \log n)$ messages even in the worst case, which is optimal for rings. Time: messages travel one hop at a time, so an election takes $O(n)$ sequential hops, plus a timeout for each dead node that must be skipped.

Pros:

- Each node communicates only with its neighbor, so the load per node is low, and some variants are proven message-optimal.
- The algorithms are simple and their correctness is well established.

Cons:

- Messages travel one hop at a time, so election time grows with the size of the ring.
- A silent node breaks the ring. Its predecessor must detect the failure and skip it, adding a timeout at every dead hop.
- The ring structure has to be repaired whenever a node leaves or rejoins.
- Selection is based on fixed IDs, so the algorithm does not naturally account for current load.

4.3 Spanning-tree and echo-based election

How it works: In networks with an arbitrary topology, election can be reduced to building a spanning tree, a subset of links that connects every node without cycles. In the Gallager–Humblet–Spira algorithm (1983), nodes repeatedly merge small fragments into larger ones until a single minimum spanning tree remains. The two nodes on the final "core" edge then pick one of themselves as leader, so election comes as a by-product of building the tree. Echo (convergecast) algorithms work in two waves. The initiator floods an exploration message outward, which forms a tree. Each node then reports the best candidate value in its subtree back toward the root, and the root broadcasts the winner down the tree.

Cost: Gallager–Humblet–Spira needs $O(n \log n + |E|)$ messages, where $|E|$ is the number of links. A single echo wave needs about $2|E|$ messages and time proportional to the network's diameter. With several initiators competing, the cost grows, because weaker waves must be suppressed.

Pros:

- Works on any network topology, not only rings or complete graphs.

- Asymptotically efficient in messages for large, sparsely connected networks.

Cons:

- Considerably more complex to implement and verify than Bully or ring algorithms.
- After a failure, the tree must be partly or fully rebuilt before another election can complete.
- Brings no benefit to RescueMesh, where three nodes are already directly connected to each other.

4.4 Flooding and complete-network algorithms

How it works: In FloodMax (Lynch, 1996), each node repeatedly sends the highest ID it knows to all its neighbors. After as many rounds as the network's diameter, every node knows the global maximum, and that node is the leader. For complete networks, where every node is directly linked to every other, Korach–Moran–Zaks (1984) and Afek–Gafni (1991) reduce the cost by letting candidates capture other nodes in rounds, so weaker candidates drop out early. Randomized algorithms (Kutten et al., 2015) go further by contacting only a sample of nodes.

Cost: FloodMax sends O(diameter × |E|) messages and needs diameter rounds. The complete-network algorithms need O(n log n) messages, and the randomized ones need fewer than n messages with high probability. In a three-node complete network, the diameter is 1, so FloodMax reduces to a single all-to-all exchange: 6 messages and one round trip.

Pros:

- In a small complete network, election completes in one round trip.
- Every node sees every candidate's value, so all nodes reach the same decision independently.
- Any comparable value can be flooded in place of IDs, including a load score; this is the basis of our MWBE design.

Cons:

- Assumes synchronous rounds, so delayed messages can cause nodes to decide on different values.
- Flooding wastes messages in large or densely connected networks.
- The message savings of the complete-network and randomized algorithms only matter at scales far beyond three nodes.

4.5 Randomized election (Itai–Rodeh, 1990)

How it works: When nodes have no unique IDs, there is nothing to compare, so each node draws a random number and the highest wins. If two or more nodes tie, only the tied nodes draw again, and this repeats until a single winner remains. Raft uses the same idea in its election timeouts: each node waits a random time before starting an election, so two nodes rarely start at once and split the vote.

Cost : Each round costs about the same as a ring election, and only a few rounds are needed on average. The number of rounds is not fixed, but the election ends with probability 1.

Pros:

- Works even when nodes have no unique IDs.
- Simple to implement.

Cons:

- Election time is only probabilistic, with no fixed upper bound.
- Unnecessary when nodes already have unique IDs, as in RescueMesh; we borrow only the randomized back-off.

4.6 Election inside consensus protocols

How it works: In replicated systems, the leader orders every update so that all copies stay identical, and electing it is part of the consensus protocol itself. These protocols share one core idea: every leadership period carries a number (a term, epoch, ballot or view), and a leader needs support from a **majority** of nodes. Two leaders of the same period therefore cannot both succeed,

and messages carrying an old number are rejected. The main protocols differ in how they pick the leader:

- **Paxos / Multi-Paxos** (Lamport, 1998 and 2001): one proposer acts as leader, chosen with a failure detector. Two competing leaders cannot cause wrong results, because ballot numbers make stale proposals fail; they only slow progress.
- **Raft** (Ongaro and Ousterhout, 2014): a node that stops hearing heartbeats waits a random timeout, starts a new term, and asks for votes. Each node votes once per term, and only for a candidate whose state is at least as up to date as its own; a majority of votes wins. With **Pre-Vote**, a node first checks that it could win, so a returning node cannot disrupt a healthy leader.
- **Zab** (ZooKeeper; Junqueira et al., 2011): votes are ranked by epoch, then by the latest update seen, then by server ID, so the most up-to-date server wins.
- **Viewstamped Replication** (Oki and Liskov, 1988; Liskov and Cowling, 2012): leadership rotates in a fixed order, so when the leader fails, the next node in line takes over without a vote.

Cost: One election round needs O(n) messages: a vote request to each node and a reply. Failover time is dominated by the detection timeout plus one round trip.

Pros:

- Majority agreement prevents two leaders from acting at once, even during a partition or while a node is silenced.
- Term and epoch numbers block a stale leader from issuing commands when it returns.
- Raft and Zab elect the most up-to-date node, which simplifies recovery.
- Widely used, formally verified, and well documented.

Cons:

- Needs a majority: if 2 of 3 nodes are down, no leader can be elected. This is acceptable for us, since only one node is silenced at a time.
- The leader is chosen by timing or state freshness, not by load.
- Failover speed depends heavily on how the timeouts are tuned.

- A full implementation with a replicated log is a large effort and could hide the mechanisms the course assesses.

Note that the leader only has to order updates, while other nodes can still do the actual work. Single-leader consensus therefore does not rule out load balancing; its real costs are an extra hop through the leader and implementation complexity.

4.7 Metric-based and priority-based election

How it works: Instead of electing the node with the highest fixed ID, these approaches elect the node that performs best right now, based on measurements such as CPU load, queue length or network delay. The approaches differ in where the measurements enter the election:

- **Performance-aware candidate selection (SEER):** the system predicts which node will perform best and lets only that node run for leader.
- **Multi-policy voting (Raft-PLUS):** nodes vote using several ranking policies, and "opposition" rules force a leader to step down when its performance degrades.
- **Node-priority timers:** higher-priority nodes get shorter election timeouts, so they usually start, and win, the election first.
- **Performance-keyed Bully and ring variants:** the classic algorithms, but comparing node and link performance instead of IDs (Section 5).

Cost: The same as the underlying algorithm, plus the cost of collecting measurements. That overhead can be close to zero when measurements are piggybacked on heartbeats that are sent anyway.

Pros:

- Directly supports load balancing, because the least busy or best-connected node is chosen.
- Adapts automatically when nodes differ in capacity or become temporarily overloaded.

Cons:

- Measurements can be outdated or noisy, so the "best" node may be misjudged.

- Nodes holding different measurements may disagree on the winner.
- Tie-breaking and stability rules are needed to stop leadership from switching back and forth on small fluctuations.
- Many published evaluations use only simulation, not real deployments.

4.8 Leases and fencing

How it works: A **lease** gives a leader authority for a fixed period. The leader must renew it before it expires, and if it fails or loses contact, the lease runs out and authority lapses automatically. A **fencing token** is a number, such as an epoch or term, attached to every command the leader sends. Each time a new leader is chosen, the number increases, and nodes reject any command carrying an older number. A returning leader that still believes it is in charge is therefore ignored.

Cost: No extra messages while the lease is valid, because renewal can ride on heartbeats. After a leader fails, the system must wait for the lease to expire before a new leader can act.

Pros:

- Makes leadership a time-limited claim that any node can check, rather than something nodes must assume.
- The standard protection against a stale leader returning after a failure, which is exactly RescueMesh's silenced-node case.
- A natural fit for Phase 2 resource reservations, which also need to expire and be checkable.

Cons:

- Relies on node clocks running at roughly the same rate.
- Lease length is a trade-off: a long lease means a longer outage after a leader fails, while a short lease means more frequent renewals.

5. Recent Literature

Recent IEEE and ACM work concentrates on three themes that matter directly to RescueMesh: **faster, adaptive failover**, **behavior under partial (omission) failures**, and **choosing the leader by measured quality**. The table is ordered newest first.

Work	Venue	Contribution	Relevance to RescueMesh
Dynatune: Dynamic tuning of election parameters for timely leader failover in SMR	IEEE Access, 2026 (extends an IPDPSW 2025 paper)	Tunes heartbeat interval and election timeout from RTT and packet-loss measurements piggybacked on heartbeats. Reports leader-failure detection time cut by about 80% and out-of-service time by about 45% versus stock Raft, with no extra messages.	Direct template for our adaptive failure detector; we piggyback load metrics on the same heartbeats
Li, Xu, Zhou, Wang, Zhang and Crowcroft, Distributed consensus network: a modularized communication framework and reliability probabilistic analysis	IEEE Transactions on Networking 34, 2026	Decomposes Raft, Paxos, PBFT and HotStuff into communication components and computes consensus reliability with link loss and node failure as probabilities	Gives a method to reason about how message loss affects our election rounds
SCRaft: Achieving fast and stable elections in Raft	IEEE conference, 2025 (IEEE Xplore	The current leader pre-designates a successor during its term, cutting split votes and election latency after a leader failure	Same idea as our ranked fallback chain: the next executor is

	doc. 11083 162)		known before the failure
The study of a distributed networking method based on an improved Raft protocol	ACM conference proceedings, 2024	Node-priority election: higher-priority nodes get shorter election countdowns	Shows a priority key can replace first-come voting with little protocol change
Brief announcement on the message complexity of fault-tolerant leader election and agreement; “Improved tradeoffs for leader election”	ACM ICDCN 2023; ACM PODC 2023	Message and time bounds for fault-tolerant and randomized election in complete networks	Theory background; confirms constants, not asymptotics, dominate at n = 3
Wen and Law, Delay and loss rate analysis of the log commitment process in Raft	IEEE conference, 2022	Models how packet loss stretches the expected wait for elections and commits	Supports measuring election duration under injected delay and loss
Jensen, Howard and Mortier, Examining Raft’s behaviour during partial network failures	ACM HAOC ’21 (EuroSys works	Shows etcd’s real behavior during a partial network failure differs from the textbook model and that Pre-Vote would have prevented a known outage	Strong evidence for Pre-Vote style checks so a returning silenced node cannot disrupt a

	hop), 2021		healthy coordinator
Choumas and Korakis, When Raft meets SDN: how to elect a leader over a network	IEEE NetSoft, 2020	Models Raft response time as a function of where the leader sits in the network; argues election should consider network position	Supports including client RTT in our selection score
3-Phase leader election algorithm for distributed systems	IEEE conference, c. 2019	Filter, validate and accept phases to reduce message and time cost versus Bully and Ring	Shows candidate filtering before comparison; we filter out syncing or suspected nodes

Older IEEE work that introduced ideas we reuse. "A leader election algorithm in a distributed computing system" (IEEE Xplore 525021) elects by node and link performance and pre-elects a provisional leader while the current one runs. "A new approach for election algorithm in distributed systems" (IEEE Xplore 5176072) elects a leader plus a designated alternative that takes over on failure. Howard et al.'s Raft Refloated (ACM SIGOPS OSR, 2015) independently reproduced Raft's election-time measurements and showed how timeout choice affects convergence under contention.

Non-IEEE/ACM origins, cited for specific ideas. SEER (arXiv 2104.01355) predicts the best-performing replica and assigns it the candidate role. Raft-PLUS (MDPI Symmetry, 2022) replaces first-come voting with four election policies and adds three "opposition" policies that make a degraded leader step down; it reports about 40% higher write throughput than Raft when leader network quality degrades.

What the literature tells us

1. The strongest recent practical gains come from **adaptive timeouts** and **performance-aware selection**, not new message patterns.

2. **Partial failures** (a node that is alive but unreachable) are where real systems break, and Pre-Vote plus epoch fencing is the accepted fix.
3. **Pre-designated successors** (SCRaft, the leader-alternative scheme) shorten failover; our ranked fallback chain applies this per request.
4. Almost no published work elects a node **per request**, which is our setting. We must therefore evaluate it empirically ourselves (Section 7.8).

6. Comparative Summary

No surveyed technique is best at both jobs: a metric-keyed all-to-all exchange wins for per-request selection, and a quorum-based, epoch-fenced election wins for the coordinator. The comparison below uses exactly the five criteria the project specification asks for.

6.1 Comparison on the five required criteria

Technique	Assumptions	Message complexity	Failure behavior	Convergence	Suitability for RescueMesh
Bully	Synchronous timing; accurate timeout-based failure detection; totally ordered IDs	$n - 1$ best, $O(n^2)$ worst	Tolerates crashes during election; a returning high-ID node seizes leadership; two coordinators possible under partition or silence	1 timeout + 1–2 message delays	Good for per-request selection if keyed on load; unsafe alone as coordinator

Chang–Roberts ring	Logical unidirectional ring; successor failure detection	O(n log n) average, $O(n^2)$ worst	A silent node breaks the ring; repair needs a skip-timeout per dead hop	O(n) sequential hops	Poor: hop latency on every request and fragile under 20 s silences
Hirschberg–Sinclair / Peterson	Ring (bidirectional or unidirectional)	O(n log n) worst	Same as ring	O(n) hops	Poor: no benefit at n = 3
Spanning tree (GHS) / echo	Arbitrary connected graph	O(n log n + E)	Tree must be rebuilt after failure	High	Overkill for a complete graph of 3 nodes
FloodMax / all-to-all exchange	Complete graph; bounded delay	$n(n-1)$ per round	Silent node simply contributes no value	1 round trip	Best for per-request selection with a load score; gives no agreement guarantee for a coordinator
Randomized (Itai–Rodeh)	Anonymous nodes	Expected O(n) per round	Not addressed	Probabilistic	Not needed (we have IDs); borrow back-off only

Raft election	Partial synchrony; majority quorum; durable term and vote	O(n) per round	Majority prevents split-brain; pre-vote and terms fence a returning node	Randomized timeout + 1 round trip	Strong for coordinator; poor for per-request selection
Paxos / Multi-Paxos	Partial synchrony; Ω for liveness	O(n) per phase	Ballots fence stale leaders; safe under any asynchrony	1–2 round trips per ballot	Strong for coordinator but complex to implement
Zab fast leader election	Majority quorum	$O(n^2)$ worst	Epochs fence; most up-to-date node wins	1–2 round trips	Strong for coordinator
Viewstamped Replication	Majority quorum	O(n) per view change	View numbers fence	Detection + 1 round trip	Good for coordinator
Metric / priority variants (SEER, Raft-PLUS)	As base protocol, plus fresh metrics	As base protocol	As base; stale metrics can misrank	As base	Good: matches the spec's load criteria
Leases + fencing (add-on)	Bounded clock drift	0 extra while lease valid	Lease expires on its own;	Lease length after a failure	Essential add-on for coordinator

			epoch rejects stale commands		and Phase 2 reservations

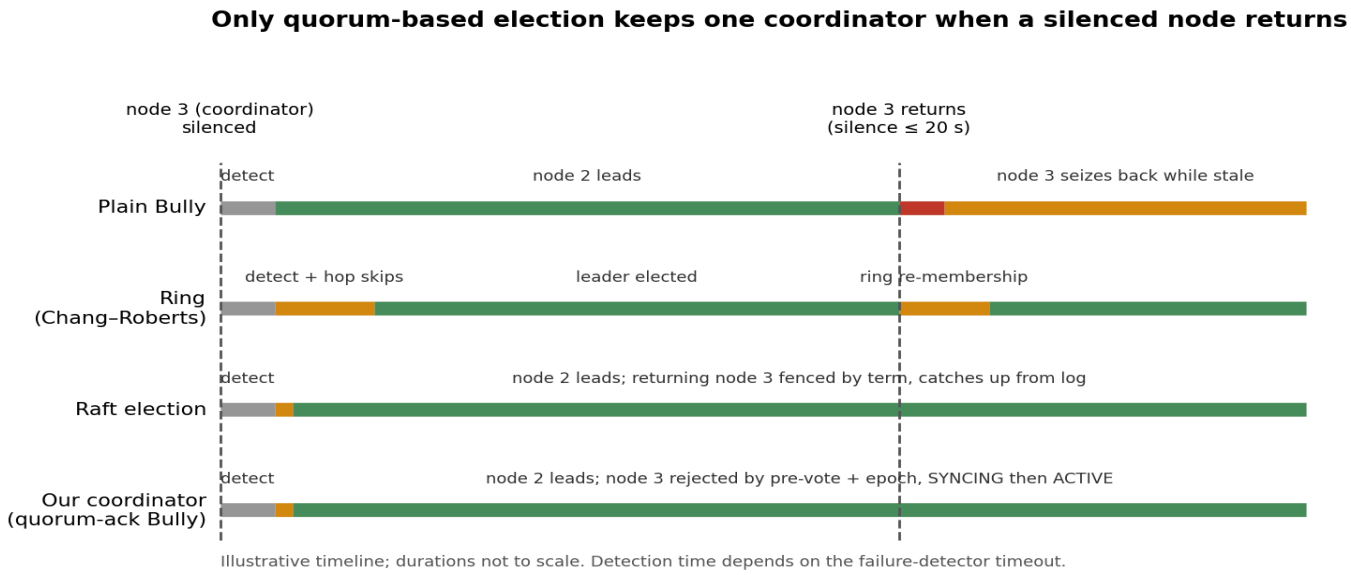
6.2 Concrete costs in the RescueMesh failure scenario (n = 3)

Asymptotic complexity barely distinguishes these algorithms with three nodes, so the table below counts messages for the actual scenario: nodes 1, 2 and 3, where node 3 is the coordinator and is silenced. Counts include messages sent to the silenced node, since the sender cannot know it is silent. T_d is the failure-detection timeout and T_e the election-reply timeout.

Algorithm	**Messages sent to elect a new coordinator**	**Time to new coordinator**
Bully, node 2 detects	3: ELECTION to node 3; COORDINATOR to nodes 1 and 3	T_d + T_e
Bully, node 1 detects	6: ELECTION to nodes 2 and 3; OK from node 2; ELECTION from node 2 to node 3; COORDINATOR from node 2 to nodes 1 and 3	T_d + T_e + 1 round trip
Ring (textbook), ring 1 → 2 → 3 → 1	6: ELECTION 1 → 2, 2 → 3 (lost), 2 → 1; COORDINATOR 1 → 2, 2 → 3 (lost), 2 → 1	T_d + 2 skip-timeouts + 4 delays
Raft without pre-vote	5: RequestVote to 2 peers; 1 vote back; first heartbeat to 2 peers	T_d (randomized) + 1 round trip
Raft with pre-vote	8: adds 2 pre-vote requests and 1 reply	T_d + 2 round trips

Our coordinator (quorum-acknowledged Bully)	8: pre-vote to 2 peers + 1 reply; ELECTION to 2 peers + 1 acknowledgment; COORDINATOR to 2 peers	T_d + 2 round trips
MWBE per request, cached scores	0 extra (CLAIM broadcast is needed anyway)	0 extra
MWBE per request, fresh bids (all 3 active)	6 BID messages	+ 1 bid window Δ

Two points stand out. First, every coordinator election costs under ten messages, so message complexity is not a deciding factor at our scale. Second, the quorum-based options pay one extra round trip but are the only ones that stay correct when the silenced node returns, as the timeline below shows.



Illustrative timeline of the silenced-coordinator scenario under four election schemes

Under plain Bully, the returning node immediately reclaims leadership while its state is stale. The ring must re-admit the returning node and repair its successor links. Raft and our quorum-acknowledged coordinator both reject the returning node's claim, because its epoch or

term is out of date, and the leadership elected during the outage stays in place while the returning node catches up.

Conclusions that drive the design

1. **At n = 3, asymptotics do not discriminate.** Every candidate costs at most a handful of messages. Round trips per decision, behavior under the injected failure, and the selection key decide the choice.
2. **Per-request selection needs a dynamic key and one round trip.** An all-to-all exchange of load scores with a Bully-style ID tie-break meets both, with 6 messages at n = 3, or none if scores ride on heartbeats.
3. **The coordinator needs quorum and fencing, not just highest-ID.** Plain Bully is unsafe against a silenced node that returns believing it still leads. Requiring acknowledgment from a majority (2 of 3) and stamping every coordinator action with an epoch removes the split-brain risk.
4. **Failure detection quality bounds everything.** Adaptive timeouts (φ-accrual or Dynatune-style) are needed because CPU-heavy planning jobs delay heartbeats.

7. Projected Techniques for RescueMesh and Justification

We will implement two cooperating mechanisms. **MWBE** picks the executor for every request in at most one round trip, using a load score with a Bully-style ID tie-break. A **quorum-acknowledged, epoch-fenced Bully election** picks a stable coordinator that a silenced node cannot usurp when it returns.

7.1 Requirements and their election implications

Requirement	Source in project spec	Implication for election
Peers choose the executor of each request	Client multicasts; peers elect the worker	Fast, deterministic per-request decision
Measurable selection criteria	Queue length, CPU, I/O, RTT, reliability	Dynamic, multi-factor key

Transparency	Client never knows the executor or the substitute	The substitute must be determined without client involvement
One node silenced for up to 20 s	Remaining peers continue service	Fast detection; in-flight work taken over
Duplicates harmless	Idempotent retries required	Two executors must never produce conflicting results
Revived node catches up first	Recovery requirement	Syncing nodes are ineligible for any role
Peers choose the node to silence	Fault-injection requirement	A single, agreed coordinator
Conflicting claims (Phase 2)	Leases and reservations	Epoch-fenced authority for arbitration

7.2 Per-request executor: Metric-Weighted Bid Election (MWBE)

MWBE is a Bully election in which the comparison key is a measured load score instead of a static ID, run as a one-round all-to-all exchange because the cloud is a complete graph of three nodes. It is our own adaptation, not a library feature.

Score. Each node maintains exponentially weighted moving averages (EWMA) of its queue length q, CPU utilization c, mean recent service time s, RTT to the requesting client r, and recent failure or timeout rate f. Each is normalized to [0, 1] and combined, lower being better:

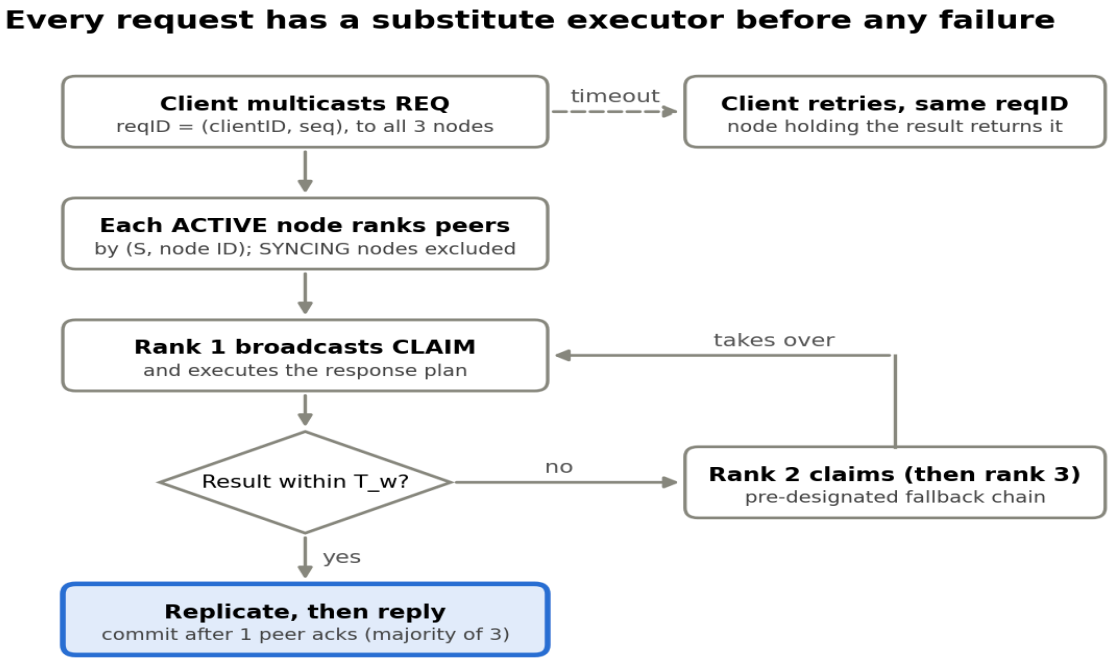
$S_i = w_q \cdot q_i + w_c \cdot c_i + w_s \cdot s_i + w_r \cdot r_{i,client} + w_f \cdot f_i$, where all inputs are normalized to [0, 1] and the weights sum to 1

Scores are quantized (for example to steps of 0.05) so that measurement noise does not reorder nodes that are effectively equal. Ties are broken by node ID, which is the Bully priority, making the ranking a total order.

Protocol for one request

1. **Request.** The client middleware multicasts REQ(reqID, payload) to all three nodes, where reqID = (clientID, sequence number) is unique. Each node records reqID in a deduplication table. This follows the multicast navigation pattern of Coulouris et al. (§13.2), in which a client multicasts to a group of servers and the appropriate one replies; RescueMesh applies it to request assignment rather than name resolution.
2. **Rank.** Each ACTIVE node ranks all ACTIVE peers by (S, nodeID). Suspected or SYNCING nodes are excluded. Two variants will be compared in Phase 1: **(A) fresh bids**, where each node broadcasts BID(reqID, S) and waits a bid window Δ (6 extra messages, one round trip); and **(B) cached scores**, where each node uses the scores piggybacked on the latest heartbeats (no extra messages, slightly staler).
3. **Claim.** The rank-1 node broadcasts CLAIM(reqID, S, nodeID, epoch) and starts executing. If two nodes claim the same request because their views differed, the claim with the lower (S, nodeID) wins and the other aborts. Every node can apply this rule locally to the two claims it sees.
4. **Execute and replicate.** The executor computes the response plan from a versioned snapshot of incident and resource state, replicates the result to its peers, and replies to the client once at least one peer has acknowledged (a majority including itself).
5. **Fallback.** Ranks 2 and 3 form a **fallback chain**, the per-request version of a pre-designated successor. If rank 2 sees neither CLAIM progress nor a result within an adaptive watchdog time T_w (expected service time plus a safety margin), it claims the request itself, and likewise for rank 3.
6. **Client retry.** Independently, if the client gets no reply within its timeout it re-multicasts the request with the **same reqID**. A node that already holds the committed result returns it; otherwise the request re-enters the protocol. Retries use exponential backoff with randomized jitter, so clients that time out together do not retransmit together.
7. **Duplicate safety.** Results are stored by reqID and the first committed result wins. Because planning is deterministic for a given snapshot version, a duplicate execution either produces the same plan or is discarded.

The fallback chain is related to the backup-request technique of Dean and Barroso (2013), who show that a delayed backup request can cut tail latency at modest cost. Here the ranking already computed for the request provides the backup without another selection round.



MWBE request flow with fallback chain and client retry

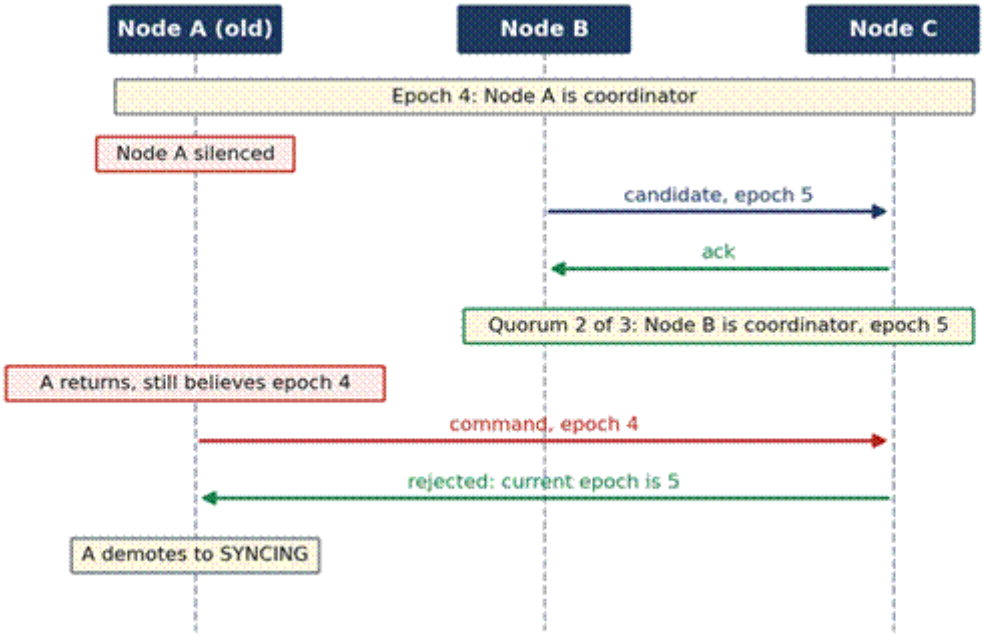
The executor, its substitutes and the client's retry are all fixed before any failure, so a silenced node never stalls a request.

7.3 Stable coordinator: quorum-acknowledged, epoch-fenced Bully

A plain Bully coordinator is unsafe against our failure model. A silenced node keeps its memory and may return still believing it is coordinator, while the other two have elected someone else. We therefore add the three safety tools from Section 2.5, borrowed from Raft but without its replicated log.

1. **Durable epoch.** Each node persists its current epoch and the candidate it acknowledged in that epoch before replying to anyone.

2. **Pre-vote.** A node that suspects the coordinator first asks its peers whether they also suspect it. It proceeds only if at least one peer agrees, so a node that was merely silenced cannot start a disruptive election on return.
3. **Bully ordering with quorum.** The candidate with the highest (state version, node ID) among the reachable nodes increments the epoch and sends ELECTION(epoch, version). A peer acknowledges only if the epoch is newer than any it has seen, it has not acknowledged another candidate in that epoch, and the candidate's state is at least as current as its own. **Acknowledgment from 2 of 3 nodes** (including itself) is required before broadcasting COORDINATOR(epoch).
4. **Fencing.** Every coordinator command carries its epoch; peers reject any command with an older epoch.
5. **Self-fencing lease.** The coordinator keeps its role only while a majority acknowledges its heartbeats within a lease period L. If it loses majority contact, it steps down on its own.



The lease mechanism follows Gray and Cheriton (1989), and using sequence numbers to neutralize a process that pauses and later resumes follows the Chubby lock service (Burrows, 2006).

Coordinator duties: choosing the node to silence and the outage length (up to 20 s) with a seeded random generator, which also makes experiments repeatable; designating the recovery donor for a rejoining node; and, in Phase 2, arbitrating lease and reservation conflicts with epoch-fenced tokens. The coordinator never sits on the per-request path, so it is not a throughput bottleneck.

7.4 Failure detection, takeover and rejoin

Detection. Nodes exchange heartbeats every 100–200 ms carrying their EWMA metrics and state version. Suspicion uses a φ-accrual detector, or Dynatune-style timeouts derived from measured RTT and loss, so that heartbeats delayed by CPU-heavy planning do not trigger false failovers.

Node states. Each node is ACTIVE, SUSPECTED (by peers), or SYNCING. Only ACTIVE nodes are ranked by MWBE or eligible as coordinator.

State	Ranked by MWBE?	Eligible as coordinator?	Entry condition
ACTIVE	Yes	Yes	Applied version matches the donor's committed version, and the node completes one heartbeat round with both peers.
SUSPECTED	No	No	Failure detector crosses the suspicion threshold.
SYNCING	No	No	Node observes silence beyond the threshold and demotes itself.

Rejoin. When a silenced node resumes, it notices it has received nothing for longer than the suspicion threshold and enters SYNCING on its own. It contacts the donor named by the coordinator, adopts the current epoch (which fences any authority it held), copies a snapshot, and replays all operations after the snapshot version. It becomes ACTIVE once its applied version

matches the donor's committed version at the time of the check and it has completed one heartbeat round with both peers. Requests it receives while SYNCING are ignored, since the other two nodes are already handling them.

7.5 Why this choice

Alternative	**Why not chosen as primary**
Ring (Chang–Roberts, Hirschberg–Sinclair)	Hop-by-hop latency on every request; ring repair when a node goes silent; no dynamic key. Kept as an experimental baseline.
Spanning tree / echo	Built for arbitrary graphs; unnecessary for three fully connected nodes.
Plain Bully (static ID)	Ignores load; a returning node seizes leadership and two coordinators can coexist. Kept as a baseline and as our tie-break order.
Full Raft (or Paxos, Zab) for everything	Safe and well proven. However, routing every request through one leader adds a hop and concentrates load, and full log replication is significant effort that could hide the mechanisms being assessed. We adopt its terms, quorum acknowledgment and pre-vote for the coordinator instead.
Randomized (Itai–Rodeh)	Only needed without unique IDs; we use randomization only for back-off when elections collide.

Justification summary. MWBE uses exactly the criteria the spec lists, decides in at most one round trip, knows the substitute executor before any failure, and tolerates rare disagreement through idempotency instead of a heavy consensus round. The quorum-acknowledged coordinator closes the one safety gap MWBE leaves, split authority after a silenced node returns, at a cost of one extra round trip on rare coordinator changes.

7.6 Limitations and risks

- **No strict mutual exclusion per request.** Under message loss two nodes may briefly execute the same request. The spec permits duplicates if harmless; we will measure the duplicate-execution rate.
- **Stale or noisy metrics** can misrank nodes. Mitigations: EWMA smoothing, quantized scores, and comparing variants A and B.
- **Bid window and watchdog trade-offs.** A small Δ or T_w lowers latency but raises false takeovers; we will sweep both.
- **Heuristic weights.** Initial weights are equal and will be tuned against measured latency and fairness.
- **Majority dependence of the coordinator.** If two nodes fail at once, no coordinator can be elected. Requests still flow through MWBE, but fault injection and recovery-donor assignment pause. This is outside the spec's single-failure model.
- **Lease timing.** The coordinator lease assumes bounded clock drift between nodes; LAN clocks easily satisfy this.
    - **Slower failover under heavy load.** More queued requests and a longer catch-up for the returning node; failover and recovery are measured separately at light, medium and heavy load.

7.7 Election in the Phase 2 peer-to-peer layer

The project allows election "wherever coordination or selection of a responsible node is required." On the peer side we deliberately minimize it, because client peers join, leave and go offline far more often than cloud nodes, and every election among them would be slow and unstable.

Phase 2 need	Approach	Is an election needed?

Lease and reservation arbitration (conflicting claims)	Decided by the cloud coordinator using epoch-fenced tokens and replicated state; offline peers mark claims as speculative until reconciled	No peer election: the cloud is the single arbiter
Discovery and presence	Replicated directory in the cloud with heartbeat leases	No: any cloud node can answer queries
Incident dissemination within a zone	Bounded gossip with a hop limit (TTL), message IDs for duplicate suppression, and a fanout limit	No: gossip needs no leader
Optional zone super-peer (only if measurements show gossip overhead is too high)	Metric-keyed Bully among the zone's online peers, using uptime and connectivity as the key and a lease so the role expires if the super-peer disappears	Yes, but only as an optimization, evaluated in Phase 2

Keeping election out of the peer layer by default means every authoritative decision has one owner, the cloud coordinator, which simplifies the conflict-handling story the specification requires.

Discovery works as an attribute-based directory service rather than a name service (Coulouris et al., §13.3); entries expire if heartbeats stop, much like DNS time-to-live.

7.8 Evaluation plan for Phase 1

Automated runs of tens of thousands of requests per configuration, repeated with different seeds, comparing **MWBE-A, MWBE-B, plain Bully, and a ring baseline** under identical workloads and fault schedules. We will report:

- End-to-end latency: mean, median, p95, p99.
- Throughput as client processes increase.
- Election duration and election messages per request.
- Load distribution across the three nodes (coefficient of variation and Jain's fairness index).

- Duplicate-execution and false-takeover rates.
- Failover time: from fault injection to the first request served by a substitute.
- Recovery time: from revival to ACTIVE, under light, medium and heavy load.
- Coordinator elections per hour and how often a returning node was correctly fenced.

Jain's fairness index gives a scale-independent measure of load balance (Jain, Chiu and Hawe, 1984), and percentiles are reported because tail latency matters more to users than the mean (Dean and Barroso, 2013). Each experiment logs its random seed so any fault sequence can be replayed.

8. References

Textbooks

- Tanenbaum, A. S. and van Steen, M. *Distributed Systems*, 4th ed., 2023 (coordination chapter, election algorithms).
- Coulouris, G., Dollimore, J., Kindberg, T. and Blair, G. *Distributed Systems: Concepts and Design*, 5th ed., Addison-Wesley, 2011.
- Lynch, N. *Distributed Algorithms*. Morgan Kaufmann, 1996.

Classical papers

- Garcia-Molina, H. Elections in a distributed computing system. *IEEE Transactions on Computers* C-31(1), 1982.
- LeLann, G. Distributed systems: towards a formal approach. *IFIP Congress*, 1977.
- Chang, E. and Roberts, R. An improved algorithm for decentralized extrema-finding in circular configurations of processes. *Communications of the ACM* 22(5), 1979.
- Hirschberg, D. S. and Sinclair, J. B. Decentralized extrema-finding in circular configurations of processors. *Communications of the ACM* 23(11), 1980.
- Peterson, G. L. An O(n log n) unidirectional algorithm for the circular extrema problem. *ACM TOPLAS* 4(4), 1982.
- Dolev, D., Klawe, M. and Rodeh, M. An O(n log n) unidirectional distributed algorithm for extrema finding in a circle. *Journal of Algorithms* 3(3), 1982.
- Gallager, R., Humblet, P. and Spira, P. A distributed algorithm for minimum-weight spanning trees. *ACM TOPLAS*5(1), 1983.
- Korach, E., Moran, S. and Zaks, S. Tight lower and upper bounds for some distributed algorithms for a complete network of processors. *ACM PODC*, 1984.
- Afek, Y. and Gafni, E. Time and message bounds for election in synchronous and asynchronous complete networks. *SIAM Journal on Computing* 20(2), 1991.
- Itai, A. and Rodeh, M. Symmetry breaking in distributed networks. *Information and Computation* 88(1), 1990.

- Fischer, M., Lynch, N. and Paterson, M. Impossibility of distributed consensus with one faulty process. *Journal of the ACM* 32(2), 1985.
- Chandra, T. and Toueg, S. Unreliable failure detectors for reliable distributed systems. *Journal of the ACM* 43(2), 1996.
- Hayashibara, N., Défago, X., Yared, R. and Katayama, T. The φ accrual failure detector. *IEEE SRDS*, 2004.
- Lamport, L. The part-time parliament. *ACM TOCS* 16(2), 1998; Paxos made simple. *ACM SIGACT News* 32(4), 2001.
- Oki, B. and Liskov, B. Viewstamped replication. *ACM PODC*, 1988; Liskov, B. and Cowling, J. Viewstamped replication revisited. MIT technical report, 2012.
- Junqueira, F., Reed, B. and Serafini, M. Zab: High-performance broadcast for primary-backup systems. *IEEE/IFIP DSN*, 2011.
- Ongaro, D. and Ousterhout, J. In search of an understandable consensus algorithm. *USENIX ATC*, 2014.
- Kutten, S., Pandurangan, G., Peleg, D., Robinson, P. and Trehan, A. On the complexity of universal leader election. *Journal of the ACM* 62(1), 2015.

- Burns, J. E. A formal model for message passing systems. Technical Report TR-91, Computer Science Department, Indiana University, 1980.
- Frederickson, G. N. and Lynch, N. A. Electing a leader in a synchronous ring. *Journal of the ACM* 34(1), 1987.
- Chandra, T. D., Hadzilacos, V. and Toueg, S. The weakest failure detector for solving consensus. *Journal of the ACM* 43(4), 1996.
- Gray, C. G. and Cheriton, D. R. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. *ACM SOSP*, 1989.
- Burrows, M. The Chubby lock service for loosely-coupled distributed systems. *USENIX OSDI*, 2006.
- Dean, J. and Barroso, L. A. The tail at scale. *Communications of the ACM* 56(2), 2013.

- Jain, R., Chiu, D.-M. and Hawe, W. R. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. DEC Research Report TR-301, 1984.

Recent work (Section 5)

- Dynatune: Dynamic tuning of election parameters for timely leader failover in state machine replication. *IEEE Access*, 2026, doi:10.1109/ACCESS.2026.3724865 (arXiv:2507.15154).
- Li, Y., Xu, Z., Zhou, Z., Wang, Y., Zhang, L. and Crowcroft, J. Distributed consensus network: a modularized communication framework and reliability probabilistic analysis. *IEEE Transactions on Networking* 34, pp. 4084–4095, 2026, doi:10.1109/TON.2026.3667834.
- SCRaft: Achieving fast and stable elections in Raft consensus algorithm. IEEE, 2025, IEEE Xplore document 11083162.
- The study of a distributed networking method based on an improved Raft protocol. ACM, 2024, doi:10.1145/3701047.3701055.
- Brief announcement: On the message complexity of fault-tolerant computation: leader election and agreement. *ACM ICDCN*, 2023, pp. 294–295.
- Improved tradeoffs for leader election. *ACM PODC*, 2023.
- Wen, Y. and Law, K. L. E. Delay and loss rate analysis of the log commitment process in Raft. IEEE, 2022.
- Jensen, C., Howard, H. and Mortier, R. Examining Raft's behaviour during partial network failures. *ACM HAOC '21 (EuroSys workshop)*, 2021.
- Choumas, K. and Korakis, T. When Raft meets SDN: How to elect a leader over a network. *IEEE NetSoft*, 2020.
- 3-Phase leader election algorithm for distributed systems. IEEE, Xplore document 8819837.
- A new approach for election algorithm in distributed systems. IEEE, Xplore document 5176072.

- A leader election algorithm in a distributed computing system. IEEE, Xplore document 525021.
- Howard, H., Schwarzkopf, M., Madhavapeddy, A. and Crowcroft, J. Raft refloated: Do we have consensus? *ACM SIGOPS Operating Systems Review* 49(1), 2015.
- SEER: Performance-aware leader election in single-leader consensus. arXiv:2104.01355.
- Xu, J., Wang, W., Zeng, Y., Yan, Z. and Li, H. Raft-PLUS: Improving Raft by multi-policy based leader election with unprejudiced sorting. *Symmetry* 14(6):1122, 2022.